

Primitive Recursion on Algebraic Data Types

```
theory Primitive_Recursion
imports Main
begin
```

We can define functions recursively on algebraic data types in a way that is analogous to previous recursive functions we discussed on natural numbers or lists. Below is a function that recursively computes the set of elements in a tree, and the list of elements in the tree traversed in order.

```
fun tree_set::"'a tree  $\Rightarrow$  'a set" where
  "tree_set Leaf = {}"
| "tree_set (Node l a r) = insert a ((tree_set l)  $\cup$  (tree_set r))"

definition tree_eg where "tree_eg  $\equiv$  Node (Node Leaf 1 (Node Leaf 2 Leaf)) (4::nat) (Node Leaf 3 Leaf)"

value "tree_set tree_eg"

fun inorder::"'a tree  $\Rightarrow$  'a list" where
  "inorder Leaf = []"
| "inorder (Node l a r) = (inorder l) @ [a] @ (inorder r)"

value "inorder tree_eg"

end
```